设计系统集体 · 下列的

会员专享故事

你的设计系统存在人工智能依赖问题，只是你还没意识到而已。（即使在人类看来一切正常）

阅读时间：5分钟 · 9小时前



乔治·威廉·阿马兰 跟随

听 分享 更多的

为什么自主编码工具在“足够好”的组件库上会失败

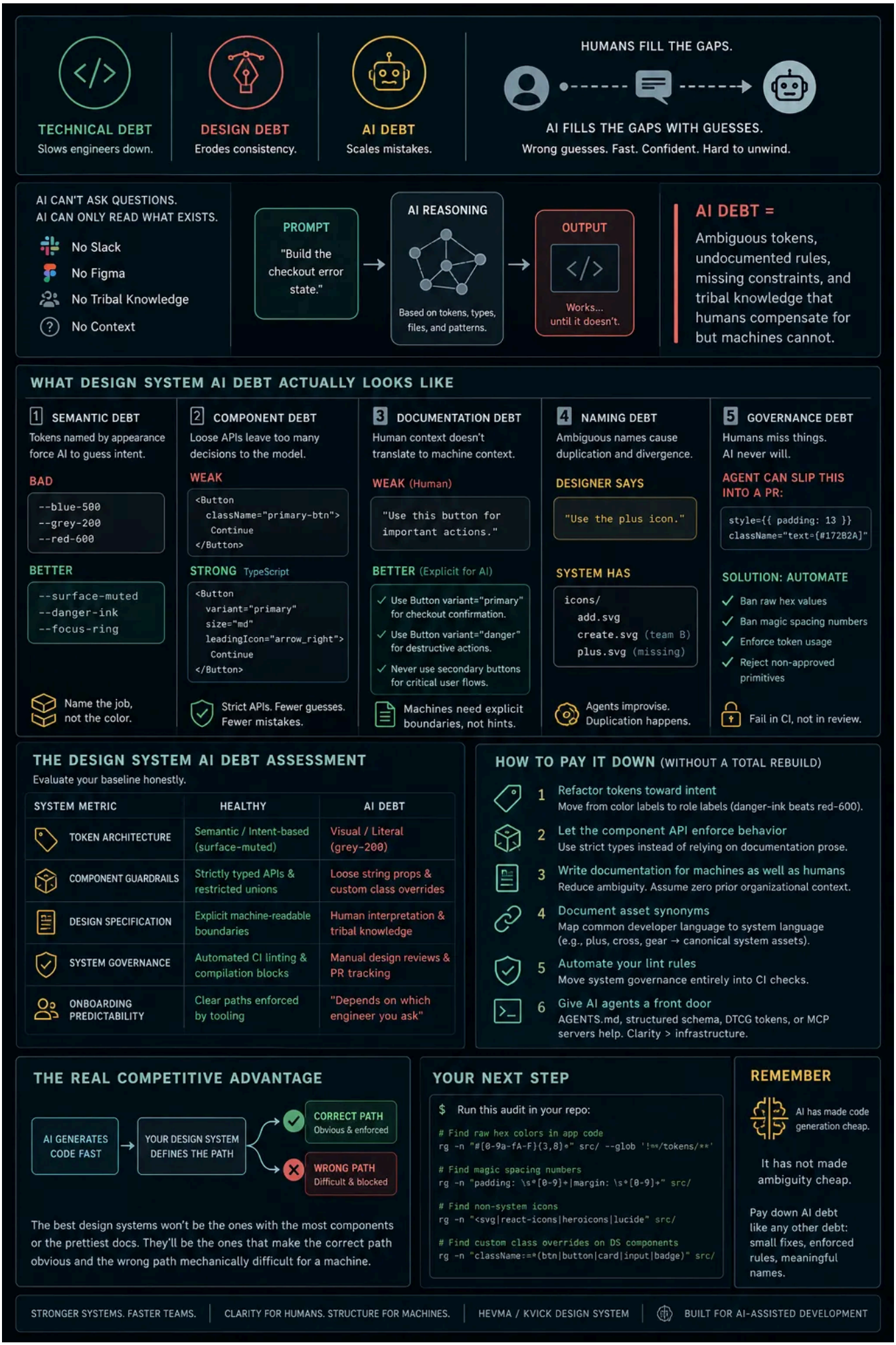
技术债务会拖慢工程师的工作效率。

设计缺陷会削弱一致性。

设计系统人工智能的弊端有所不同。它会放大错误。

大多数平台团队并未意识到他们的设计系统存在人工智能依赖，因为这套系统对人类开发人员来说仍然有效。工程师知道在 Slack 中哪里可以找到答案。设计师知道该使用哪些变体。高级工程师知道应该避免使用哪些组件快捷方式。

非会员好友链接


TECHNICAL DEBT
Slows engineers down.
DESIGN DEBT
Erodes consistency.
AI DEBT
Scales mistakes.
HUMANS FILL THE GAPS.
AI FILLS THE GAPS WITH GUESSES.
Wrong guesses. Fast. Confident. Hard to unwind.
AI CAN'T ASK QUESTIONS.
AI CAN ONLY READ WHAT EXISTS.
No Slack
No Figma
No Tribal Knowledge
No Context
PROMPT
"Build the checkout error state."
AI REASONING
Based on tokens, types, files, and patterns.
OUTPUT
Works... until it doesn't.
AI DEBT =
Ambiguous tokens, undocumented rules, missing constraints, and tribal knowledge that humans compensate for but machines cannot.
WHAT DESIGN SYSTEM AI DEBT ACTUALLY LOOKS LIKE
1 SEMANTIC DEBT
Tokens named by appearance force AI to guess intent.
BAD
--blue-500
--grey-200
--red-600
BETTER
--surface-muted
--danger-ink
--focus-ring
Name the job, not the color.
2 COMPONENT DEBT
Loose APIs leave too many decisions to the model.
WEAK
<Button
className="primary-btn">
Continue
</Button>
STRONG TypeScript
<Button
variant="primary"
size="md"
leadingIcon="arrow_right">
Continue
</Button>
Strict APIs. Fewer guesses. Fewer mistakes.
3 DOCUMENTATION DEBT
Human context doesn't translate to machine context.
WEAK (Human)
"Use this button for important actions."
BETTER (Explicit for AI)
Use Button variant="primary" for checkout confirmation.
Use Button variant="danger" for destructive actions.
Never use secondary buttons for critical user flows.
Machines need explicit boundaries, not hints.
4 NAMING DEBT
Ambiguous names cause duplication and divergence.
DESIGNER SAYS
"Use the plus icon."
SYSTEM HAS
icons/
add.svg
create.svg (team B)
plus.svg (missing)
Agents improvise. Duplication happens.
5 GOVERNANCE DEBT
Humans miss things. AI never will.
AGENT CAN SLIP THIS INTO A PR:
style={{ padding: 13 }}
className="text-[#172B2A]"
SOLUTION: AUTOMATE
Ban raw hex values
Ban magic spacing numbers
Enforce token usage
Reject non-approved primitives
Fail in CI, not in review.
THE DESIGN SYSTEM AI DEBT ASSESSMENT
Evaluate your baseline honestly.
SYSTEM METRIC
HEALTHY
AI DEBT
TOKEN ARCHITECTURE
Semantic / Intent-based (surface-muted)
Visual / Literal (grey-200)
COMPONENT GUARDRAILS
Strictly typed APIs & restricted unions
Loose string props & custom class overrides
DESIGN SPECIFICATION
Explicit machine-readable boundaries
Human interpretation & tribal knowledge
SYSTEM GOVERNANCE
Automated CI linting & compilation blocks
Manual design reviews & PR tracking
ONBOARDING PREDICTABILITY
Clear paths enforced by tooling
"Depends on which engineer you ask"
HOW TO PAY IT DOWN (WITHOUT A TOTAL REBUILD)
1 Refactor tokens toward intent
Move from color labels to role labels (danger-ink beats red-600).
2 Let the component API enforce behavior
Use strict types instead of relying on documentation prose.
3 Write documentation for machines as well as humans
Reduce ambiguity. Assume zero prior organizational context.
4 Document asset synonyms
Map common developer language to system language (e.g., plus, cross, gear → canonical system assets).
5 Automate your lint rules
Move system governance entirely into CI checks.
6 Give AI agents a front door
AGENTS.md, structured schema, DTCG tokens, or MCP servers help. Clarity > infrastructure.
THE REAL COMPETITIVE ADVANTAGE
AI GENERATES CODE FAST
YOUR DESIGN SYSTEM DEFINES THE PATH
CORRECT PATH
Obvious & enforced
WRONG PATH
Difficult & blocked
The best design systems won't be the ones with the most components or the prettiest docs. They'll be the ones that make the correct path obvious and the wrong path mechanically difficult for a machine.
YOUR NEXT STEP
$ Run this audit in your repo:
Find raw hex colors in app code
rg -n "#[0-9a-fA-F]{3,8}\b" src/ --glob '!**/tokens/**'
Find magic spacing numbers
rg -n "padding: \s*[0-9]+|margin: \s*[0-9]+" src/
Find non-system icons
rg -n "<svg|react-icons|heroicons|lucide" src/
Find custom class overrides on DS components
rg -n "className=.*(btn|button|card|input|badge)" src/
REMEMBER
AI has made code generation cheap.
It has not made ambiguity cheap.
Pay down AI debt like any other debt: small fixes, enforced rules, meaningful names.
STRONGER SYSTEMS. FASTER TEAMS.
CLARITY FOR HUMANS. STRUCTURE FOR MACHINES.
HEVMA / KVICK DESIGN SYSTEM
BUILT FOR AI-ASSISTED DEVELOPMENT

人类会填补这些空白，人工智能则不会。

相反，它会用猜测来填补这些空白。当这些猜测出错时，它们速度很快、信心十足，而且在任何人注意到之前就能传播到几十个生产文件中。

这就是设计系统人工智能债务。它指的是一系列模糊不清的标记名称、未记录的组件规则、缺失的约束以及人类每天都在弥补但机器无法弥补的经验知识。

人工智能编码工具的兴起并非造成这些架构缺陷，而是暴露了这些缺陷。

人类可以提问，而人工智能只能读取你的代码。

多年来，设计系统一直可以做到*几乎*透明。

- 一个名为 `blue-500` .的令牌
- `add` 一个名为“.”的图标
- 故事书自述文件上写着*“运用常识”*。
- 一个组件有十五个可选属性，但没有任何关于何时使用它们的指导。

人类开发者可以暂停代码、打开 Figma、搜索 Slack、询问队友或依靠工程经验。而人工智能代理则无法做到这一点。它只能根据代码、类型和上下文中明确存在的信息进行推理。

如果设计意图不明确，模型就会进行即兴发挥。而这种以机器速度进行的即兴发挥，在收到拉取请求之前，看起来效率极高。

设计系统人工智能债务的真实面貌是什么样的

1. 语义债务

许多系统仍然展示视觉描述，而不是结构化的标记意图。

- **坏的：** `--blue-500` ， `--grey-200` ， `--red-600`
- **更好的：** `--surface-muted` ， `--danger-ink` ， `--focus-ring`

人类能够理解这两种含义。当标记名称描述用途而非外观时，人工智能的性能会显著提升。我们的目标并非追求更美观的名称，而是消除歧义，避免语言逻辑模型（LLM）将文本标记映射到布局边界。

2. 组件债务

人工智能能够理解应用程序接口（API），但它无法理解隐藏在以往评审会议中的设计者意图。

- 虚弱的：`<Button className="primary-btn">`
- 强的：
- TypeScript
- `<Button variant="primary" size="md" leadingIcon="arrow_right">`

在应用中打开 ↗

Medium

这是设计运维中最容易被忽视的人工智能债务形式之一。组件文档通常假定存在人为上下文：
> *“使用此按钮执行重要操作。”*

人类开发者理解“重要”在你的产品语境中意味着什么。模型则不理解。

* **更清晰明确的系统指令：**
    * *使用 `Button variant="primary"` 进行结账确认。*
    * *使用 `Button variant="danger"` 进行破坏性操作。*
    * *切勿在关键用户流程中使用辅助按钮。*

人类依靠推理。**机器需要明确的界限。**

4. 命名债务
这种现象无处不在，严重阻碍了库的使用。一位设计师说：“使用加号图标。”

设计系统资源库中包含 `add` 图标。另一个产品团队构建了一个名为 `create` 的自定义图标。当 AI 代理搜索“加号”时，在你的系统中找不到任何相关图标，因此它会导入一个完全独立的图标库，或者粘贴一个原始的内联 SVG 图像。这样就造成了资源重复。

问题不在于缺少资产，而在于命名歧义。

5. 治理债务
许多平台团队仍然完全依赖人工代码审查员来发现样式偏差，并将 PR 评论作为唯一的执行机制。当 UI 代码由人工逐行编写时，这种方法行之有效。但当代理同时生成大量界面代码时，这种方法就变得极其脆弱。

例如，代理人可以轻松地将这些代码插入到 400 行的 PR 中：
```tsx
style={{ padding: 13 }}
className="text-[#172B2A]"
```

解决方案并非一个庞大而复杂的平台项目。只需在设计系统软件包中内置简单的持续集成检查即可立即解决问题：

- 禁止在应用程序代码中使用原始十六进制值。
- 禁止使用魔法空格数字。
- 严格执行令牌使用规定。
- 如果存在组件，则拒绝未经批准的 HTML 基本元素。

不合规的代码应该在编译器级别自动报错，而不是等待人工审核。

设计系统人工智能债务评估

向您的平台团队提出以下五个问题，并诚实地评估您的基准：

System Metric	Healthy	AI Debt
Token Architecture	Semantic / Intent-based (`surface-muted`)	Visual / Literal (`grey-200`)
Component Guardrails	Strictly typed APIs & restricted unions	Loose string props & custom class overrides
Design Specification	Explicit machine-readable boundaries	Human interpretation & tribal knowledge
System Governance	Automated CI linting & compilation blocks	Manual design reviews & PR tracking
Onboarding Predictability	Clear paths enforced by tooling	"Depends on which engineer you ask"

系统中“AI债务”指标越多，说明你的库就越会阻碍工程团队的自动化速度。

如何偿还债务（无需彻底重建）

解决这个问题不需要进行为期六个月的大规模变革。从小处着手，循序渐进：

- **将标记重构为意图**：从颜色标签移至角色标签（ `danger-ink` 节拍 `red-600` ）。
- **让组件 API 强制执行行为**：使用严格的类型定义，而不是仅仅依赖文档文字。
- **编写文档时既要面向机器，也要面向人**：减少歧义。假设读者对组织背景一无所知。

- **文档资产同义词**：将通用开发人员语言映射到系统语言（例如，将 `plus` 、`cross` 或 `gear` 直接映射到您的规范系统资产）。
- **自动化你的 lint 规则**：尽可能将系统治理完全转移到 CI 检查中。
- **给 AI 代理一个入口**：组件库中的文件 `AGENTS.md` 、结构化的模式布局、DTCG 令牌或本地 MCP 工具都可以提供帮助。

归根结底，具体的基础设施远不如结构清晰度重要。

真正的竞争优势

人工智能使代码生成成本变得异常低廉，**但并没有使结构性歧义变得廉价**。

在这个新格局下，能够成功的系统设计并非拥有最多自定义组件或最精美文档的系统，而是那些其API能够清晰地展现正确执行路径，并让机器难以识别错误路径的系统。

多年来，人类一直在弥补设计系统中的不清晰之处。人工智能只是将这种人为弥补的程度暴露无遗，揭示了其背后究竟有多少不足之处。

这就是为什么人工智能债务对平台工程至关重要。它并非一种新的债务类型——**而是早已存在的系统债务，如今终于显现出来**。

> 加入讨论：您的设计系统是针对人类推理还是机器执行进行了优化？目前，当 AI 编写代码时，您在 UI 代码库中遇到的最大架构瓶颈是什么——是标记、组件属性还是资源命名？欢迎在下方讨论。

如果这引起了共鸣……

👏如果这篇文章让你重新思考如何发布 AI 工具，**请鼓掌——它确实能帮助更多人找到这篇文章**。

💬请在评论区分享你的团队对人工智能产品失去信任的那一刻。我会阅读每一条评论。

🔁把它分享给下周即将发布“完美人工智能”的工程师、项目经理或领导。

保持联系

如果你是人工智能产品的开发者、领导者或投资者，我会定期撰写这方面的文章。

🌐网站： georgeamalan.com 💼 **LinkedIn**： linkedin.com/in/amalan-william

关注我们，了解更多关于软件工程、设计系统和人工智能系统工程、规范工具包以及在真实组织中工程系统和交付人工智能的复杂现实。

用户体验设计 设计系统 人工智能 人工智能 网站开发



下列的

发表于设计系统联盟

5.6K 位粉丝 · 最后发布时间： 9 小时前

这是一个热情友好的社区，面向所有热爱可扩展、一致性设计的设计师和开发者。在这里，您可以浏览文章、获取洞见和资源，构建并完善您的设计系统。加入我们，交流学习，共同塑造系统化设计的未来。



跟随

乔治·威廉·阿玛兰编剧

108 位关注者 · 307 位关注者

对科技充满好奇，同时又秉持人文关怀。感恩所学，并铭记每个系统背后的人们。https://www.georgeamalan.com

No responses yet



Duan

What are your thoughts?